Loading by default

If you are frequently loading other template libraries using the `{% load %}` tag, there is a shortcut. Inside the `TEMPLATES` configuration setting, there is a section called `OPTIONS`. Inside `OPTIONS`, you can add a key called `builtins` with a list as its value. Whatever tag modules are named in this list are automatically loaded across all your templates. For example, to use the `static` template tag library throughout your system, modify `TEMPLATES` in settings.py as follows:

```
TEMPLATES = [
    {
        ...
        "OPTIONS": {
            "builtins": ["django.templatetags.static"],
            ...
        },
    },
]
```

7.4 Uploads

HTML forms allow your users to attach files, submitting them to your site. Unlike other submitted data, files generally shouldn't go in a database; it is far more efficient to store them as files on the server. Files often get associated with data, though, so you need database columns to reference them. Django handles this through the `FileField` model field.

I've mentioned before that you should never trust your users; for file uploads, this goes double. Just because the user says they're uploading an image doesn't mean what they're uploading is an image. At least for the case of images, Django can help you here. The `ImageField` inherits from the `FileField` and validates that the associated file is an image. To do this, it uses the Pillow image library (https://pillow.readthedocs.io/en/latest/), which has to be installed separately:

```
(venv) RiffMates$ python -m pip install pillow
```

7.4.1 Configuring your project for uploads

Like static files, uploaded files shouldn't be served by Django. In production, these files should be handled by a web server. In development, you can add a couple lines of code to get the Django dev server to serve files. Also like static files, you need a place to store the uploaded files and configuration to tell Django where that is.

Where you keep your uploaded files is a matter of preference. Unlike static files, uploaded files should not be part of your project. For smaller projects and testing, the pattern I typically use is a directory named *outside* at the same level as the project directory. Within the outside folder, I create another directory with the same name as the project. I started this habit when working with smaller customers, who ran multiple Django projects from the same server. Having the project name inside the outside directory created a separate upload location for each project.